**Escape Sequences**

" " or ' ' print('') or print("")

```
print(''Hello World'')

'Hello World'

print('"Hello World"')

"Hello World"

print(''hello world'')

  Cell In[3], line 1
    print(''hello world'')
          ^
SyntaxError: invalid syntax. Perhaps you forgot a comma?


print(""Hello"")

  Cell In[4], line 1
    print(""Hello"")
          ^
SyntaxError: invalid syntax. Perhaps you forgot a comma?


# Single quote \n
print('\'Hello World\'')

'Hello World'

# \n New Line
print("Hello \nWorld")

Hello
World

#t tab
print("Name: Ahmed")

Name: Ahmed

print("Name:\t Ahmed")

Name:  Ahmed
```

**Variables**

reserved memory location to store values

```
print(25)
```

```
25

TD = 1580 #m
Res_Depth = 1040
KB = 35

print(TD)
print(Res_Depth)
print(KB)

1580
1040
35

print(TD-KB)

1545

TD_TVDSS = TD-KB
print(TD_TVDSS)

1545

TD = 2200 #m
Res_Depth = 2080
KB = 35

print(TD)
print(Res_Depth)
print(KB)

2200
2080
35

Float = 2.45
Complex = 6+3j

print(Float+Complex)

(8.45+3j)

Well, TD = "Well_1", 2500
print(Well)
print(TD)

Well_1
2500
```

**Rules for Varaibles nomenclature**

```
#Start with letter or _

Fm_1 = 'Pliocene'

_Fm_1 = "Oligocene"


# Can not start with a number

9_FM = "Pliocene"
```

```
  Cell In[19], line 3
    9_FM = "Pliocene"
     ^
SyntaxError: invalid decimal literal
```

```
# Contain A-Z , 0-9, _

Fm-1 = "Pliocene"
```

```
  Cell In[20], line 3
    Fm-1 = "Pliocene"
    ^
SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?
```

```
#Variable names are case sensitive

gr = 50
Gr = 60
GR = 70

print(gr)
```

```
50
```

```
print(GR)
```

```
70
```

```
total_GR = gr + Gr + GR
print(total_GR)
```

```
180
```

**Strings**

```
Well = "GS-197"
Field = "GOS"
Country = "Egypt"
```

```
Info = Well + Field + Country
print(Info)
```

```
GS-197GOSEgypt
```

**String Concatenation**

```
print(Well + " " + Field + " " +Country)
```

```
GS-197 GOS Egypt
```

```
Well = "GS-197 "
Field = "GOS "
Country = "Egypt "

print(Well+Field+Country)
```

```
GS-197 GOS Egypt
```

```
print(Well + 1970)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[31], line 1
----> 1 print(Well + 1970)

TypeError: can only concatenate str (not "int") to str
```

```
print(Well + True)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 print(Well + True)

TypeError: can only concatenate str (not "bool") to str
```

```
print(Well + " " + "1970")
```

```
GS-197  1970
```

```
print(Well + " " + str(1970))
```

```
GS-197  1970
```

**String Multiplication**

```
print(Well+3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[35], line 1
----> 1 print(Well+3)

TypeError: can only concatenate str (not "int") to str
```

```
print(Well * 3)
```

```
GS-197 GS-197 GS-197
```

**String Formatting**

```
field = 'KG Basin'
press = 5000 #psi

print('reservoir pressure is'  + press + " " 'psi')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[37], line 4
      1 field = 'KG Basin'
      2 press = 5000 #psi
----> 4 print('reservoir pressure is'  + press + " " 'psi')

TypeError: can only concatenate str (not "int") to str
```

```
print('reservoir pressure is '  + str(press) + " " +'psi')
```

```
reservoir pressure is 5000 psi
```

```
print('reservoir pressure is {} psi'.format(press))
```

```
reservoir pressure is 5000 psi
```

```
print(f'resrvoir pressure is {press} psi')
```

```
resrvoir pressure is 5000 psi
```

```
print('reservoir pressure  in {} is {} psi'.format(field, press))
```

```
reservoir pressure  in KG Basin is 5000 psi
```

```
print(f'reservoir pressure in {field} is {press} psi ')
```

```
reservoir pressure in KG Basin is 5000 psi
```

```
print('reservoir pressure  in {} is {} psi'.format(press, field))
```

```
reservoir pressure  in 5000 is KG Basin psi
```

```
print('reservoir pressure  in {field_name} is {pressure}
psi'.format(pressure = press, field_name = field))
```

```
reservoir pressure  in KG Basin is 5000 psi
```

```
print(f'reservoir pressure in {field} is {press+100} psi ')
```

```
reservoir pressure in KG Basin is 5100 psi
```

**Input Function**

```
input("What is the Field name? ")
```

```
'Belayim Land'
```

```
field_name = input("What is the Field name? ")
```

```
print(field_name)
```

```
Belayim Land
```

```
no_of_wells = input('What is number of wells? ')
print(no_of_wells)
```

```
195
```

```
print(f'This field has {no_of_wells} wells ')
```

```
This field has 195 wells
```

```
type(no_of_wells)
```

```
str
```

```
Top_of_Nubia = input('Top of Nubia ? ')
Base_of_Nubia = input('Base of Naubia')
print(Top_of_Nubia)
print(Base_of_Nubia)
```

```
3200
3600
```

```
Nubia_Thickness = Base_of_Nubia - Top_of_Nubia
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[56], line 1
```

```
----> 1 Nubia_Thickness = Base_of_Nubia - Top_of_Nubia

TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

```
Nubia_Top_Base = Top_of_Nubia + Base_of_Nubia

print(Nubia_Top_Base)
```

```
32003600
```

```
Top_of_Nubia = int(input('Top of Nubia ? '))
Base_of_Nubia = int(input('Base of Naubia'))
print(Top_of_Nubia)
print(Base_of_Nubia)

Nubia_Thickness = Base_of_Nubia - Top_of_Nubia
print(Nubia_Thickness)
```

```
3200
3600
400
```

```
Top_of_Nubia = input('Top of Nubia ? ')
Base_of_Nubia = input('Base of Naubia')
print(Top_of_Nubia)
print(Base_of_Nubia)
```

```
3200
3600
```

```
Nubia_Thickness = int(Base_of_Nubia) - int(Top_of_Nubia)
print(Nubia_Thickness)
```

```
400
```

```
Field, Main_Reservoir = input("What is the field name and main
reservoir? ")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call
last)
Cell In[64], line 1
----> 1 Field, Main_Reservoir = input("What is the field name and main
reservoir? ")

ValueError: too many values to unpack (expected 2)
```

```
Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split()
```

```
print(Field)
print(Main_Reservoir)
```

```
BM
Belayim
```

```
Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call
last)
Cell In[67], line 1
----> 1 Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split()

ValueError: too many values to unpack (expected 2)
```

```
Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split(',')
```

```
print(Field)
print(Main_Reservoir)
```

```
Belayim Marine
Belayim
```

```
Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split(',')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call
last)
Cell In[70], line 1
----> 1 Field, Main_Reservoir = input("What is the field name and main
reservoir? ").split(',')

ValueError: not enough values to unpack (expected 2, got 1)
```

**String Indexing**

```
var = 'Reservoir'
```

```
var[0]
```

```
'R'
```

```
var[3]
```

```
'e'
```

```
var[-1]
```

```
'r'
```

```
var[9]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[75], line 1
----> 1 var[9]

IndexError: string index out of range
```

```
len(var)
```

```
9
```

```
var[8]
```

```
'r'
```

**String Slicing**

```
print(var)
```

```
Reservoir
```

```
var[1:3]
```

```
'es'
```

```
var[:5]
```

```
'Reser'
```

```
var[3:]
```

```
'ervoir'
```

```
var[1:-3]
```

```
'eserv'
```

```
# Variable [Start_Index : End_index : step argument]
```

```
var[:6:1]
```

```
'Reserv'
```

```
var[:6:2]
```

```
'Rsr'
```

```
var[::-1]
```

```
'riovreseR'
```

```
var[::-2]
```

```
'rorsR'
```

**String Methods**

```
# Lower method .lower()  convert all characters to lowercase
field = 'Abu RuDEis SiDri'
field.lower()
```

```
'abu rudeis sidri'
```

```
field
```

```
'Abu RuDEis SiDri'
```

```
field_lower = field.lower()
field
```

```
'Abu RuDEis SiDri'
```

```
field_lower
```

```
'abu rudeis sidri'
```

```
field_upper = field.upper()
field_upper
```

```
'ABU RUDEIS SIDRI'
```

```
field_title = field.title()
field_title
```

```
'Abu Rudeis Sidri'
```

```
#count
field.count('i')
```

```
3
```

```
field_lower.count('i')
```

```
3
```

```
field_upper.count('i')
```

```
0
```

```
field_title.count('I')
```

```
0
```

```
#Split

.split('Spearator', 'maxsplit')
field_lower
```

```
'abu rudeis sidri'
```

```
field_lower = 'abu rudeis,sidri'
field_lower
```

```
'abu rudeis,sidri'
```

```
field_1, field2 = field_lower.rsplit(",", 1)
print(field_1)
print(field2)
```

```
abu rudeis
sidri
```

```
#strip

log = input("What is the log ? ")

print(log)
```

```
                    sonic
```

```
log_new = log.strip()

log_new
```

```
'sonic'
```

```
#Replace method

#.replace('original chcacters', 'chaaracters to be replaced')

Acoustic_Impedance = "The product of sonic and density"

Acoustic_Impedance = Acoustic_Impedance.replace('sonic', 'velocity')

Acoustic_Impedance
```

```
'The product of velocity and density'
```

```
#find

drill_mud = 'Drilling mud is used to lubricate the bit. This mud also
helps us to achieve pressure overbalance. Drilling mud also helps in
lifting drill cuttings'
```

```
drill_mud.find('mud')
```

```
9
```

```
drill_mud.find('mud', 49)
```

```
108
```

```
#Strings are Immutable

field_lower = "abu rudeis sidri"

field_lower
```

```
'abu rudeis sidri'
```

```
field_lower.find('r')
```

```
4
```

```
field_lower[4]
```

```
'r'
```

```
field_lower[4] = "R"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[131], line 1
----> 1 field_lower[4] = "R"

TypeError: 'str' object does not support item assignment
```

```
field_lower.replace('r', 'R', 1)
```

```
'abu Rudeis sidri'
```

```
field_lower

'abu rudeis sidri'

field_lower_R = field_lower.replace("r", "R", 1)

field_lower_R

'abu Rudeis sidri'

field_lower_R[4] = 'r'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
```

```
last)
Cell In[135], line 1
----> 1 field_lower_R[4] = 'r'

TypeError: 'str' object does not support item assignment
```